

Lists & Tuples

Storing and Organizing Multiple Values

Session 5

What We're Covering Today

- Data types & data structures — primitive vs. non-primitive
- What is a list? Indexing, negative indexing & advanced slicing
- append(), extend(), insert() — including negative indices
- Removing items: pop() — three ways — & remove()
- Reordering: sort() vs. sorted(), and reverse()
- min(), max(), sum(), membership, equality & nested lists
- Lists as stacks, and a taste of time complexity
- Tuples — the list that cannot change
- Tuple packing, unpacking & nested tuples
- Two mini projects tying everything together



PART 0

Data Types & Data Structures

A little vocabulary before we touch a single list.

What Is a Data Type?



A label for a single value

- A data type tells Python what kind of value something is — a number, some text, true/false, and so on
- It also decides what you're allowed to do with that value — you can add two numbers, but not two booleans the same way
- You already met four of these in an earlier session: int, float, str, and bool

What Is a Data Structure?



A way to hold many values at once

- A data structure organizes and stores multiple values together, so you can manage them as one group
- Instead of creating 100 separate variables for 100 student names, a data structure holds all 100 in one place
- Every data structure is built out of data types — a list of scores is a structure full of int values
- Lists and tuples, today's whole topic, are both data structures



Primitive vs. Non-Primitive — The Big Split



Primitive Types

- Hold exactly one single value
- Simple, fixed in size, stored directly
- Examples: int, float, bool



Non-Primitive Types

- Can hold many values, or a whole collection
- Built out of primitive types (or even other non-primitives)
- Examples: list, tuple, dictionary, array

Primitive Types — Across Languages

Concept	Python	C	Java
Whole number	int	int	int
Decimal number	float	float / double	float / double
True or False	bool	no true bool (0/1)	boolean
Single character	no separate type	char	char

Non-Primitive Types — Across Languages

Concept	Python	C	Java
Ordered collection	list	array	ArrayList / array
Fixed collection	tuple	const array	final array
Key-value pairs	dict	no built-in (struct)	HashMap
Text	str	char array	String (an object!)

Python's Twist: Everything Is an Object



Even a single number is an object

- In C and Java, primitive types are raw values sitting directly in memory — fast and simple, but rigid
- In Python, even a simple integer is secretly an object with extra information attached to it
- This is part of why Python feels more flexible and consistent, but runs a little slower than C
- Proof you've already seen: type() works on every single value, even numbers — because everything truly is an object

Where Do Strings Fit?



A little bit of both worlds

- Strings hold text, but they are technically an ordered sequence of characters, not one indivisible value
- That makes a string feel simple like a primitive, but structured enough to be indexed and sliced — just like a list
- We will go deeper on strings in an upcoming session — for today, just know they blur the line a little

Python's Data Structures — What's Coming



List

Today



Tuple

Today



String

Already introduced



Dictionary

Coming soon



Set

Coming soon

Quick Recap: Primitive vs. Non-Primitive



int

Primitive



float

Primitive



bool

Primitive



list

Non-Primitive



tuple

Non-Primitive



dict / set

Non-Primitive



PART 1

What Is a List?

A List Is a Row of Numbered Boxes



Ordered. Changeable. Mixed types allowed.

- A list stores several values together, in one single variable
- Each box has a position, called an index, starting at 0 — not 1
- A list can hold any data type, even mixed together in the same list
- Unlike a tuple (later today), a list can be changed after it's created



Creating a List

- Square brackets [ ] create a list
- Items are separated by commas
- This exact list — mixing int, int, and str — is completely valid

```
l1 = [7, 9, "harry"]
print(l1)

print(type(l1))   # <class 'list'>
```



More Ways to Create a List

- The list() function builds a list from almost any other collection
- It can turn a string into a list of its characters, or a range into a list of numbers
- Multiplying an empty-ish list is a quick way to pre-fill it with repeated values

```
a = list()          # [] — an empty list, another way
b = list("abc")     # ['a', 'b', 'c']
c = list(range(5))  # [0, 1, 2, 3, 4]

d = [0] * 5         # [0, 0, 0, 0, 0]
```

List Indexing

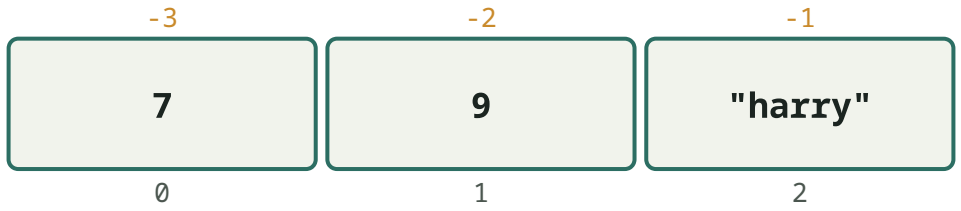
```
l1 = [7, 9, "waseem"]
```



- l1[0] → 7 (the box at index 0)
- l1[1] → 9 (the box at index 1)
- Indexes always start counting from 0, not 1 — this trips up almost everyone at first

Negative Indexing

```
l1 = [7, 9, "harry"]
```



- Negative indexes count backward from the end — -1 is always the last item
- l1[-1] → "harry" l1[-2] → 9
- Useful when you want "the last item" without needing to know the list's length



Out-of-Range: A Very Normal Error

- A list of 3 items only has valid indexes 0, 1, and 2
- Asking for an index that does not exist raises an IndexError
- This is completely normal — read the error, fix the index, move on

```
l1 = [7, 9, "harry"]

print(l1[70])
# IndexError: list index out of range
```

List Slicing

```
l1 = [7, 9, "harry"]  ->  l1[0:2]
```



- A slice l1[start:stop] grabs a whole range of items at once
- l1[0:2] → [7, 9] — starts at index 0, stops right before index 2
- The highlighted boxes are what gets included — the "stop" index itself is never included

Advanced Slicing — Negatives & Steps

- Slicing accepts negative indices too — count from the end, just like regular negative indexing
- A third number after a second colon sets a step — "every Nth item"
- A step of -1 walks backward through the whole list — a quick way to reverse it without changing the original

```
l1 = [1, 8, 7, 2, 21, 15]

print(l1[-3:-1])   # [2, 21]
print(l1[:3])      # [1, 8, 7]   (start left out = from the beginning)
print(l1[3:])      # [2, 21, 15] (stop left out = to the end)
print(l1[::2])     # [1, 7, 21]  (every 2nd item)
print(l1[::-1])    # [15, 21, 2, 7, 8, 1]  (reversed!)
```

More Ways to Access List Values

- Unpacking works on lists too, not just tuples — one value per variable, in order
- Star unpacking lets one variable "soak up" everything left over
- index() finds the position of a value, the reverse of normal indexing

```
l1 = [1, 8, 7, 2, 21, 15]

a, b, c, d, e, f = l1      # unpack all 6 into 6 variables
print(a, f)                # 1 15

first, *rest = l1
print(first)   # 1
print(rest)    # [8, 7, 2, 21, 15]

print(l1.index(21))   # 4
```

len() — How Many Items?

- len() tells you exactly how many items are in a list
- Very useful for checking a list's size before working with it

```
l1 = [1, 8, 7, 2, 21, 15]
print(len(l1))   # 6
```



LIST METHODS

Before & After — Watch the List Change

Every method below uses the same starting list: l1 = [1, 8, 7, 2, 21, 15]

.append(8) — Add to the End

```
l1 = [1, 8, 7, 2, 21, 15]  ->  l1.append(8)
```



- append() always adds the new item at the very end of the list
- The list grows by exactly one item, and nothing else moves



Does append() Take an Index?

- No — append() always adds to the end, no matter what
- It only ever accepts the value to add, never a position
- If you need a specific position, that's exactly what insert() is for, next

```
l1 = [1, 2, 3]

l1.append(4)      # fine — adds 4 to the end
l1.append(0, 4)   # TypeError — append() takes no position
```

extend() vs. append() — A Common Mix-Up

- append() adds its argument as ONE single item — even if that argument is itself a list
- extend() adds each item from another list individually, one by one
- Mixing these two up is one of the most common real bugs beginners write

```
l1 = [1, 2, 3]
l2 = [1, 2, 3]

l1.append([4, 5])
print(l1)   # [1, 2, 3, [4, 5]]   <- a nested list!

l2.extend([4, 5])
print(l2)   # [1, 2, 3, 4, 5]     <- flat, as expected
```

.insert(3, 8) — Add at a Specific Spot

```
l1 = [1, 8, 7, 2, 21, 15]  ->  l1.insert(3, 8)
```



- insert(index, value) pushes the new item into that exact position
- Everything that was already at or after that index shifts one spot to the right

insert() With a Negative Index

- insert() accepts negative indices too — they count from the end, same as everywhere else
- -1 means "insert right before the last item", not "insert at the very end"
- To truly insert at the end, append() is simpler and clearer

```
l1 = [1, 8, 7, 2, 21, 15]

l1.insert(-1, 99)
print(l1)
# [1, 8, 7, 2, 21, 99, 15]
# 99 lands BEFORE the last item, not after it
```

.pop(2) — Remove by Position

```
l1 = [1, 8, 7, 2, 21, 15]  ->  l1.pop(2)
```



l1.pop(2) removes the item at index 2, shifts everything after it left, and RETURNS the removed value → 7

- pop() is the only removal method that gives the value back to you — you can store it in a variable
- Without an index, pop() removes the last item by default

pop() — Three Ways to Call It

- pop() with no argument removes the LAST item — the most common way it's used
- pop(0) removes the FIRST item — everything after it shifts left
- pop(-1) is the same as calling pop() with nothing — explicit, but identical

```
l1 = [1, 8, 7, 2, 21, 15]

l1.pop()     # removes 15 (the last item)
l1.pop(0)    # removes 1 (the first item)
l1.pop(-1)   # same as pop() with nothing
```

.remove(21) — Remove by Value

```
l1 = [1, 8, 7, 2, 21, 15]  ->  l1.remove(21)
```



- remove() searches for the first item that matches the value you gave it, and deletes that one
- If the value appears more than once, only the first match is removed
- If the value is not found at all, Python raises a ValueError

More List Methods: index() and count()



l1.index(21)

Returns the position of the first 21 in the list

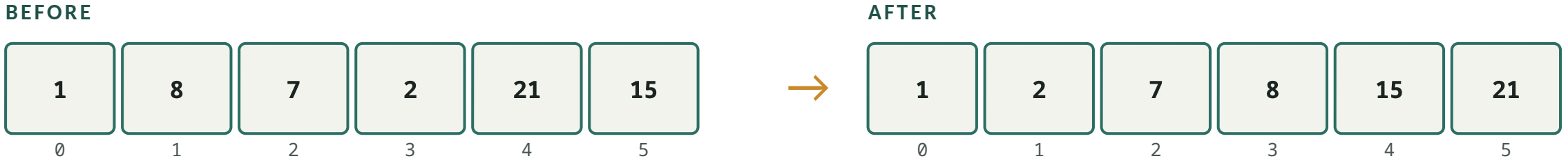


l1.count(8)

Returns how many times 8 appears in the list

.sort() — Reorder Smallest to Largest

```
l1 = [1, 8, 7, 2, 21, 15]  ->  l1.sort()
```



- sort() rearranges the list in place — the original order is gone
- By default it sorts smallest to largest — pass reverse=True for largest to smallest

sorted() vs. .sort() — Not the Same Thing

.sort() — a method

- Changes the original list directly, permanently
- Returns None — do NOT write l1 = l1.sort()
- Only works on lists

sorted() — a function

- Leaves the original list untouched
- Returns a brand new, sorted list
- Works on lists, tuples, and more

sorted() in Action

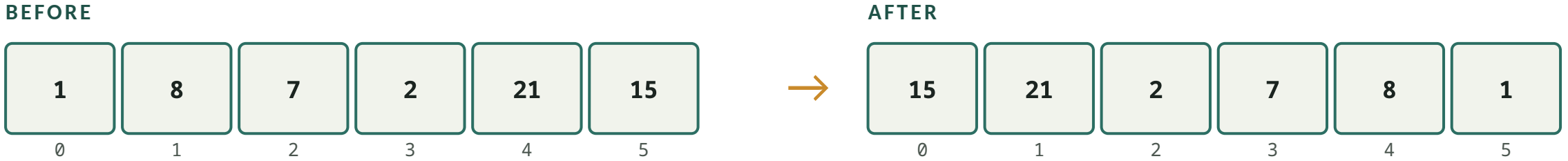
- Use sorted() whenever you want a sorted view, but need to keep the original order intact
- This is the safer default when you are not sure yet whether you'll need the original order later

```
l1 = [1, 8, 7, 2, 21, 15]

result = sorted(l1)
print(result)   # [1, 2, 7, 8, 15, 21]
print(l1)       # [1, 8, 7, 2, 21, 15]  <- unchanged!
```

.reverse() — Flip the Order

```
l1 = [1, 8, 7, 2, 21, 15]  ->  l1.reverse()
```



- reverse() simply flips the current order — it does NOT sort the values
- A list that is already sorted, once reversed, becomes largest to smallest

del and .clear() — Two More Ways to Remove

- del removes an item by index — like pop(), but does not return the value
- del can also delete the entire list at once
- clear() empties the whole list, but keeps the (now empty) list itself

```
l1 = [1, 8, 7, 2, 21, 15]
del l1[0]
print(l1)   # [8, 7, 2, 21, 15]

l1.clear()
print(l1)   # []
```

Combining Lists: + and *

- + joins two lists together into a brand new list
- * repeats a list a given number of times

```
a = [1, 2]
b = [3, 4]
print(a + b)   # [1, 2, 3, 4]

print(a * 3)   # [1, 2, 1, 2, 1, 2]
```

The Mutable Multiplication Trap

- [[]] * 3 looks like it makes 3 separate empty lists — it does not
- It actually makes 3 references to the SAME inner list
- Changing one "copy" silently changes all of them — a classic real-world and interview bug

```
grid = [[]] * 3
grid[0].append(1)
print(grid)

print(id(grid[0]))


print(id(grid[1]))


print(id(grid[2]))

# [[1], [1], [1]]   <- all three changed!


# the safe way:
grid = [[] for _ in range(3)]   # (loops covered next session)
```

The Copying Gotcha

l2 = l1 (this does NOT make a copy!)



Both names point to the exact same boxes.
Change l2, and l1 changes too.



l3 = l1.copy() -> two completely separate lists



The Shallow Copy Gotcha — Nested Lists

- .copy() only copies ONE level deep — this is called a shallow copy
- If your list contains other lists inside it, those inner lists are still shared
- Changing a nested item through the "copy" still changes the original

```
original = [1, 2, [3, 4]]
shallow = original.copy()

shallow[2].append(5)
print(original)   # [1, 2, [3, 4, 5]]  <- changed too!

# real fix: import copy; copy.deepcopy(original)
```

Shallow Copy vs. Deep Copy

- A shallow copy — .copy(), list(x), or [:] — copies the outer list only. Nested lists inside are still shared with the original
- A deep copy copies everything, all the way down. The result is completely independent, with nothing shared at all
- Python's built-in copy module provides deepcopy() for exactly this situation

```
import copy

original = [1, 2, [3, 4]]

shallow = original.copy()
deep = copy.deepcopy(original)

shallow[2].append(5)
print(original)   # [1, 2, [3, 4, 5]]  <- changed! (shared)

deep[2].append(9)
print(original)   # [1, 2, [3, 4, 5]]  <- unchanged (independent)
```

min(), max(), and sum()

- These three built-in functions work directly on a list of numbers — no method needed
- min() and max() find the smallest and largest values
- sum() adds every value in the list together

```
l1 = [1, 8, 7, 2, 21, 15]

print(min(l1))  # 1
print(max(l1))  # 21
print(sum(l1))  # 54
```

Using a List as a Stack



Last In, First Out (LIFO)

- append() + pop() together turn a plain list into a stack — a real, widely-used data structure
- The last item you added is always the first one you remove — LIFO (Last in first out) like a stack of plates
- Real examples: a browser's "back" button, and "undo" history in any app
- append() adds to the top; pop() (no argument) removes from the top



A Stack in Action

- This is the exact same append()/pop() you already know — just used with intention
- This single pattern shows up constantly in real interview questions and real applications

```
history = []

history.append("Page 1")
history.append("Page 2")
history.append("Page 3")

print(history.pop())   # "Page 3"  <- the most recent page
print(history)         # ["Page 1", "Page 2"]
```

Why Front vs. Back Matters



append() / pop()

Fast — works at the end, nothing needs to shift



insert(0, x) / pop(0)

Slower — everything after index 0 has to shift over

Checking Membership: in / not in

- You already met in and not in back in the operators session — they work on lists too
- This checks whether a value exists anywhere in the list, without needing to know its index
- Returns a plain True or False, ready to use directly inside an if

```
l1 = [1, 8, 7, 2, 21, 15]

print(7 in l1)       # True
print(100 in l1)     # False
print(100 not in l1)  # True
```

List Equality — == Compares Values

- == checks whether two lists contain the same values, in the same order
- This is different from checking if they are the literal same list in memory (that's what is is for — a topic for another day)

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)   # True  -> same values, same order
print(a == [3, 2, 1])   # False -> order matters
```

Nested Lists — A List Inside a List

- A list can hold anything — including other lists
- This is how you represent grid-like or table-like data
- Use two sets of square brackets to reach an item inside the inner list

```
grid = [[1, 2, 3], [4, 5, 6]]

print(grid[0])     # [1, 2, 3]
print(grid[0][1])  # 2
print(grid[1][2])  # 6
```



Checking for an Empty List

- An empty list is "falsy" — Python treats it as False inside a condition
- This is the cleanest, most common way to check if a list has anything in it at all

```
my_list = []

if not my_list:
    print("The list is empty")
else:
    print("The list has items")
```

Strings ↔ Lists

- list() breaks a string into a list of its individual characters
- "".join(a_list) glues a list of strings back into one single string
- This bridge between strings and lists is used constantly in real programs

```
word = "hello"
chars = list(word)
print(chars)        # ['h', 'e', 'l', 'l', 'o']

back = "".join(chars)    # " ".join(chars), ", ".join(chars)
print(back)         # "hello"
```

Converting Between List and Tuple

- list() turns almost any collection into a list
- tuple() does the same, but produces an immutable tuple instead
- Handy when you need to "unlock" a tuple temporarily, edit it as a list, then lock it again

```
a_tuple = (1, 2, 3)
a_list = list(a_tuple)
print(a_list)       # [1, 2, 3]

back_to_tuple = tuple(a_list)
print(back_to_tuple) # (1, 2, 3)
```

List Methods Cheat Sheet

append(x)

Adds x to the end

extend(list)

Adds each item from another list

insert(i, x)

Adds x at index i

pop(i)

Removes index i, returns it

remove(x)

Removes first match of x

sort()

Reorders smallest to largest, in place

reverse()

Flips the current order

index(x) / count(x)

Finds position / counts matches

copy()

Shallow copy — one level deep only

Practice Set — Lists

Try these yourself first — no code shown, that's the point

1. Create a list with your 3 favorite foods, and print it
2. Print the length of your list using len()
3. Print the item at index 1 in your list
4. Change the second item in your list to something else
5. Print the LAST item in your list using a negative index
6. Use slicing to print just the first two items
7. Check whether a value is in your list using in
8. Use sorted() on a list of numbers WITHOUT changing the original — print both



PART 2

Tuples — The List That Cannot Change

What Is a Tuple?



Ordered.
Locked.
Never changes.

- A tuple stores multiple values together, exactly like a list
- The key difference: a tuple is immutable — once created, it can never be changed
- Use a tuple for values that should stay fixed, like coordinates or days of the week
- Written with round brackets ( ) instead of square brackets [ ]



Creating Tuples

- Round brackets create a tuple
- An empty tuple is just ()
- A tuple with more than one item works exactly like a list, just with ( )

```
a = ()          # empty tuple
a = (1, 7, 2)     # tuple with 3 items

print(type(a))     # <class 'tuple'>
```



More Ways to Create a Tuple

- The parentheses are actually optional — commas alone are what make a tuple
- The tuple() function builds a tuple from almost any other collection, just like list()
- Converting a list into a tuple is a common way to "lock" it once you're done editing

```
a = 1, 7, 2          # no parentheses at all — still a tuple!
print(type(a))       # <class 'tuple'>

b = tuple()          # () — an empty tuple, another way
c = tuple([1, 2, 3])    # (1, 2, 3) — from a list
d = tuple("abc")       # ('a', 'b', 'c') — from a string
```

The Single-Element Tuple Trap



One sneaky comma

- a = (1) does NOT make a tuple — Python just sees a number in brackets
- A single-item tuple needs a trailing comma: a = (1,)
- That one comma is the only thing that tells Python "this is a tuple, not just a number"
- This is one of the most common tuple mistakes — check for the comma if something looks wrong



Tuples Are Immutable

- Trying to change an item inside a tuple raises a TypeError
- This is the whole point of a tuple — it is a promise that the data will not change

```
a = (1, 7, 2)

a[0] = 5
# TypeError: 'tuple' object does not
# support item assignment
```

Tuple Indexing & Slicing

- Indexing and slicing work exactly the same as they do for lists
- The only thing that is different about a tuple is that you cannot modify it afterward

```
a = (1, 7, 2)

print(a[0])    # 1
print(a[0:2])  # (1, 7)
```

Tuple Methods



count(x)

a.count(1) — how many times 1 appears in a



index(x)

a.index(1) — the position of the first 1 in a



Tuple Packing & Unpacking

- Packing: writing several values together, comma-separated, quietly creates a tuple
- Unpacking: assigning a tuple straight into several variables at once, in one line
- The number of variables must match the number of values exactly

```
# packing
point = 4, 7
print(type(point))   # <class 'tuple'>

# unpacking
x, y = point
print(x)   # 4
print(y)   # 7
```

Negative Indices, Slicing & Combining Tuples

- Every trick you learned for lists works on tuples too — negative indices, slicing, steps
- The only real difference: the result of slicing a tuple is still a tuple, never a list
- + and * work exactly like they do for lists

```
a = (1, 7, 2, 9, 4)

print(a[-1])      # 4
print(a[1:3])     # (7, 2)
print(a[::-1])    # (4, 9, 2, 7, 1)

print(a + (5, 6))  # (1, 7, 2, 9, 4, 5, 6)
print((1, 2) * 2)  # (1, 2, 1, 2)
```



More Ways to Access Tuple Values

- Unpacking is actually the most natural way to read a tuple's values — used constantly in real code
- Star unpacking works on tuples too, exactly like it does on lists
- index() finds a value's position, the same as it does on a list

```
point = (4, 7, 9)

x, y, z = point
print(x, y, z)      # 4 7 9

first, *rest = point
print(first)  # 4
print(rest)   # [7, 9]  <- note: rest becomes a list, not a tuple

print(point.index(7))  # 1
```

Nested Tuples

- Just like lists, a tuple can hold other tuples inside it
- Useful for representing fixed, related groups of data — like a set of coordinates

```
points = ((0, 0), (3, 4), (6, 8))

print(points[1])      # (3, 4)
print(points[1][0])   # 3
```



Lists vs. Tuples — When to Use Which



Use a List When...

- The data will change — items added, removed, or reordered
- You need methods like append(), sort(), remove()
- Example: a shopping list, or student scores you keep updating



Use a Tuple When...

- The data should never change once it is set
- You want to protect it from accidental edits
- Example: a date of birth, GPS coordinates, days of the week

Practice Set — Tuples

Try these yourself first — no code shown, that's the point

1. Create a tuple with 3 numbers
2. Use a negative index to print the last item in your tuple
3. Try changing one value in your tuple, and read the error Python gives you
4. Combine two small tuples together using +
5. Use count() to count how many times a number appears in a tuple you make
6. Pack three values into one line, then unpack them into three variables

Today's Tasks — Part A: Lists

Small tasks — do them in order

1. Write a program to sum a list with 4 numbers

2. Write a program to store seven fruits in a list entered by the user

3. Write a program to accept marks of 6 students and display them in a sorted manner

Today's Tasks — Part B: Tuples

Small tasks — do them in order

4 Check that a tuple type cannot be changed in Python — try editing one and observe the error

5 Write a program to count the number of zeros in a tuple you create with at least 3 zeros in it



PART C

Put It All Together

Two small projects using everything from this course so far — variables, operators, conditionals, and today's lists & tuples.

Mini Project 1: Class Grade Report



No loops needed — just indexing

- You have two lists: 5 student names, and their 5 matching scores
- Using direct indexing (l1[0], l1[1]...), print each student's name, score, and whether they Pass or Fail (40 or above passes)
- Print the highest score using max(), and the class average using sum() and len()
- Bonus: use sorted() to also show the scores ranked, without changing your original list



Mini Project 2: Contact Card



Ties together tuples + f-strings

- Store a friend's name, age, and phone number together in a single tuple — this is personal data that shouldn't accidentally change
- Print a formatted "contact card" sentence using all three values together
- Try to update their age directly inside the tuple, and let Python's error prove why a tuple was the right choice here
- Then show the correct way to "update" it: build a brand new tuple with the changed age



RECAP

A list can grow, shrink, and change.
A tuple, once made, stays exactly as it is.

Next session: loops — now that lists exist, this is where they finally pay off.